

AttackPilot 设计说明书

2026 年第三届全国大学生数智链应用大赛
区块链大类 技术应用—应用实践与应用设计

作品名称	AttackPilot
所选分类	技术应用—应用实践与应用设计
学校名称	中山大学
团队队长	待填写
团队成员	待填写
指导教师	待填写
提交日期	2026 年待填写

本说明书介绍 AttackPilot 的应用背景、设计理念、开发过程、区块链技术应用、作品特色与不足，并给出可复核的部署和效果材料。

1. 作品概述

1.1 创作背景

区块链交易和智能合约执行过程公开可查，但真实安全事件往往由准备、触发、套利和清理等多笔交易共同组成。安全分析人员需要在区块浏览器、Trace 工具、合约源码、模拟环境和日志之间反复切换，才能回答资金如何被转移、漏洞位于哪段逻辑以及修复是否有效。AttackPilot 面向这一具体痛点，将交易收据、事件日志、资金变化、执行 Trace 和合约源码统一为可追溯的证据链，并以多智能体协作推进分析和验证，降低链上攻击复盘的时间成本和理解门槛。

1.2 作品简介

AttackPilot 是面向链上安全事件的攻击复盘 Web 应用。用户可以输入单笔或多笔交易 Hash，也可以选择 SushiSwap、ApeCoin 等缓存案例。系统依次完成链上取证、攻击检测、跨交易上下文分析、动态 Trace 探索、根因定位、补丁生成和补丁重放验证，并将关键证据、Agent 状态、任务日志、攻击路径和结构化报告集中展示。平台还提供漏洞教学、案例知识库、报告导出和取证助手，面向安全分析人员、合约开发者和学习研究人员使用。

1.3 目标受众与使用场景

安全分析人员用于事故初筛、攻击路径复盘和证据核验；合约开发者用于定位根因、查看修复建议并检查补丁能否阻断原攻击；学习研究人员用于通过真实案例理解漏洞机制、链上信号和修复思路。

2. 设计理念

2.1 设计主题与创意来源

作品以“证据驱动的链上攻击复盘”为主题，方法基础来自团队 TracePilot 研究工作。创意来源于安全专家在实际复盘中不断提出假设、查找证据、运行验证并修正判断的工作方式。平台把这一闭环过程显式化，使模型推理不脱离交易事实和执行结果。

2.2 核心设计思路

系统按“交易取证—智能体分析—补丁验证—证据归档”组织工作流。宏观分析智能体识别交易关系、地址角色和资金流向；Trace 智能体围绕可疑节点选择性展开；补丁智能体生成修复建议并在模拟环境中重放攻击；审查智能体检查证据链和结果一致性。完成的报告进入知识库，为后续分析和教学提供可复用上下文。

2.3 作品主要功能

功能模块	主要能力
交易复盘	输入单笔/多笔交易 Hash，选择真实缓存案例，执行链上取证。
攻击检测与定位	识别攻击角色、资金流、攻击阶段，并将根因定位到函数和业务规则。
补丁生成与验证	生成修复建议，编译并重放原始攻击，反馈验证结果。
结果复核与交互	展示 Agent 状态、证据、攻击路径、日志、报告和取证助手。
学习与知识沉淀	整理真实案例的背景、PoC、链上信号和修复思路，支持检索。

3. 开发过程

阶段	主要工作
研究与问题定义	围绕 DApp 攻击复盘梳理跨交易故障定位、动态 Trace 和补丁验证需求。
后端与智能体	实现 FastAPI 服务、任务调度、MCP 工具、链上数据获取、Agent 状态和结构化报告。
前端工作台	实现交易输入、案例选择、任务控制台、攻击路径、Agent 日志、报告和教学视图。
数据与部署	接入 PostgreSQL、Redis、Docker Compose，准备 SushiSwap、ApeCoin 等缓存案例。
验证与整理	通过报告、日志、补丁重放和前端构建检查流程，并整理部署说明与演示材料。

3.2 所使用的技术和系统架构

视图层使用 React 18、Vite 5 和 TypeScript 5；业务逻辑层使用 FastAPI、Pydantic、Uvicorn、WebSocket 和 MCP 工具集；多智能体负责宏观交易分析、Trace 调试、根因定位、补丁生成、重放验证和结果审查；数据层使用 PostgreSQL 15 保存任务、报告和结构化数据，Redis 7 缓存高频日志和上下文；链上数据通过 JSON-RPC、Etherscan 和 Tenderly 获取或模拟。Docker Compose 负责后端、PostgreSQL 和 Redis 的本地编排。

4. 区块链技术应用

4.1 使用的区块链平台和版本

作品面向 Ethereum/EVM 兼容链上的真实交易数据，使用 JSON-RPC 获取交易和状态，使用 Etherscan 类接口获取合约源码与 ABI，使用 Tenderly 进行交易模拟和补丁重放。作品不是另行发行代币或搭建新链，而是把区块链公开账本、智能合约执行轨迹和模拟环境用于安全分析应用。

4.2 区块链主要存储的信息类型

平台主要读取并组织交易 Hash、区块和交易收据、事件日志、调用关系、资金变化、合约地址、函数签名、源码位置、状态读写、攻击阶段和补丁重放结果。任务状态、报告和日志索引保存在本地 PostgreSQL/Redis 中；链上原始证据仍以公开链上数据和对应接口返回结果为依据。

5. 作品特色与创新

5.1 作品创新点

第一，支持跨交易攻击复盘，把准备、触发、套利和清理阶段组织成连续攻击路径。第二，采用动态 Trace 探索，在减少噪声和上下文压力的同时保留关键调用、状态和资金证据。第三，把补丁生成与攻击重放验证纳入同一闭环，使结论不仅来自模型推理，还能接受执行层面的检查。第四，把完成的复盘报告沉淀为可检索知识，服务后续分析、开发者修复和安全教学。

5.2 与同类作品的区别与优势

传统工具通常提供交易浏览、Trace 展示或静态候选位置，用户仍需人工串联证据。AttackPilot 以任务为中心统一交易输入、智能体过程、证据审查、补丁验证和报告导出，强调可追溯和可复核。项目报告中的离线评测显示，149 个真实 DApp 安全事件上，核心算法 Recall@Top-1 为 71.14%，Precision 为 77.78%；这些数字只适用于报告所述实验设置，正式提交时应以团队可核验的实验材料为准。

5.3 作品难点及解决方案

难点包括跨交易关系复杂、Trace 体量大、模型输出可能不可执行、长任务需要恢复以及证据链容易断裂。对应方案是使用角色与资金流构建交易上下文，以外部调用为骨架按需展开 Trace，用编译和模拟反馈修正补丁，使用 PostgreSQL/Redis 持久化任务和日志，并把结论绑定到交易 Hash、Trace 节点、源码位置和验证状态。

6. 创作总结

6.1 收获与经验

开发过程中形成了从研究方法到可运行平台的工程化经验：复杂分析任务需要稳定的数据模型、明确的状态边界和可恢复的任务机制；界面不只展示最终答案，还应帮助用户理解证据、过程和不确定性。

6.2 区块链使用的体会与反思

区块链数据公开、可追溯，但公开不等于易理解。交易、事件、内部调用和状态变化必须经过结构化组织才能服务于安全判断。模型可以提高阅读和归纳效率，但不应替代链上事实和可执行验证。

6.3 不足与改进方向

当前版本仍依赖外部 RPC、扫描器、模拟服务和模型接口；完整新案例分析需要相应密钥和网络条件，演示主要依赖已导入缓存案例。后续将完善多链适配、权限和密钥管理、编译器按需获取、更多可量化评测、在线部署和报告模板化导出。

7. 参考资料与第三方资源

参考资料包括 TracePilot: Self-verifiable Framework for Decentralized Applications Fault Localization across Transactions (ISSTA 2026)、FAULTSEEKER (ASE 2025)、DAppFL (ISSTA 2024)、DeFiHackLabs、Phalcon、Tenderly、Ethernaut、Solidity 官方文档以及 DeepSeek 官方模型文档。使用的前端、后端、数据库、Docker、MCP 和区块链工具均应在正式提交前按照各自许可证完成核对。

附录：成品展示与部署材料

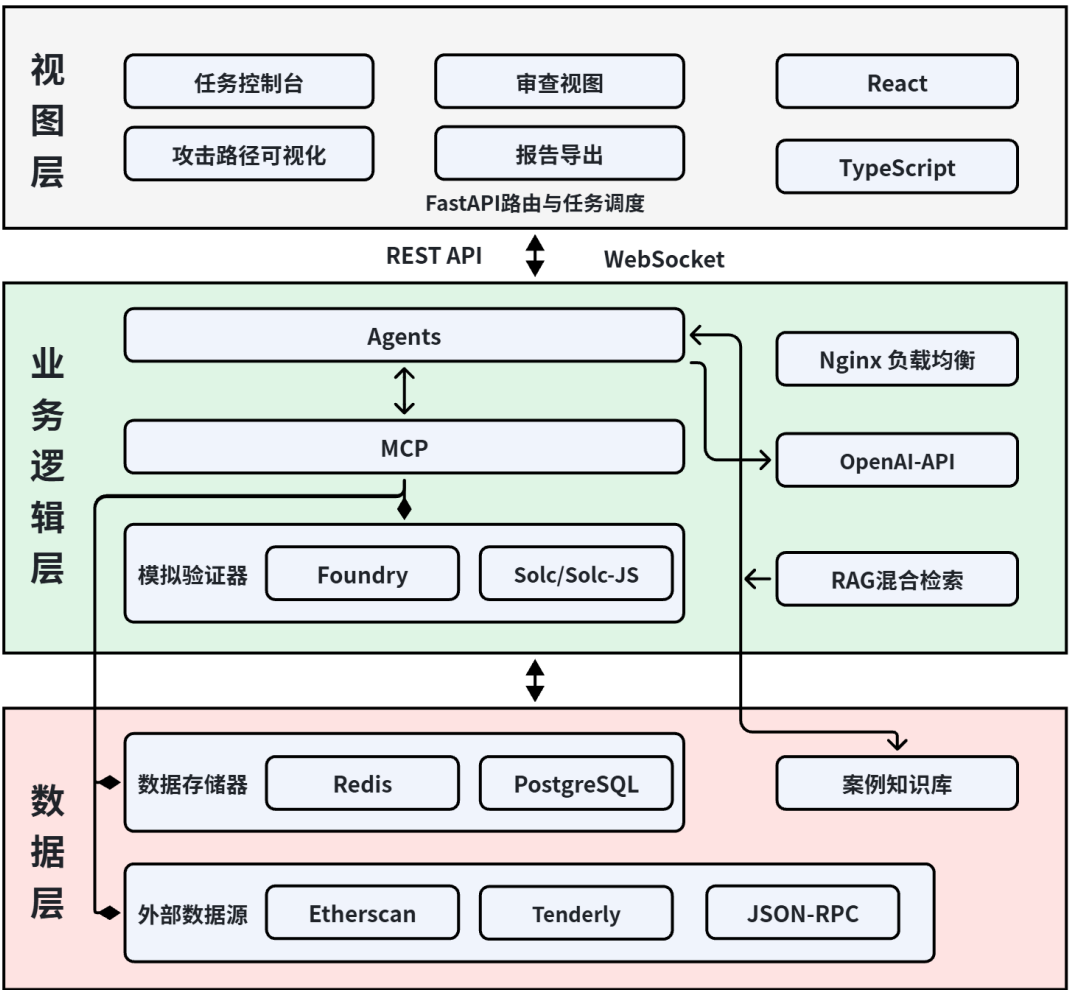


图 1 AttackPilot 技术架构

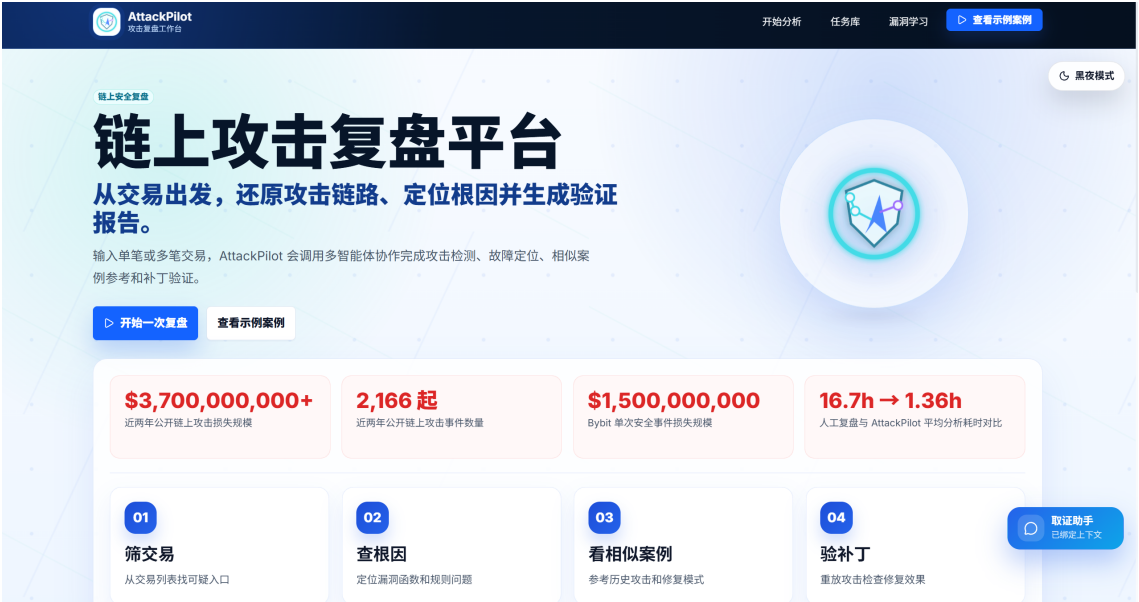


图 2 AttackPilot 产品首页

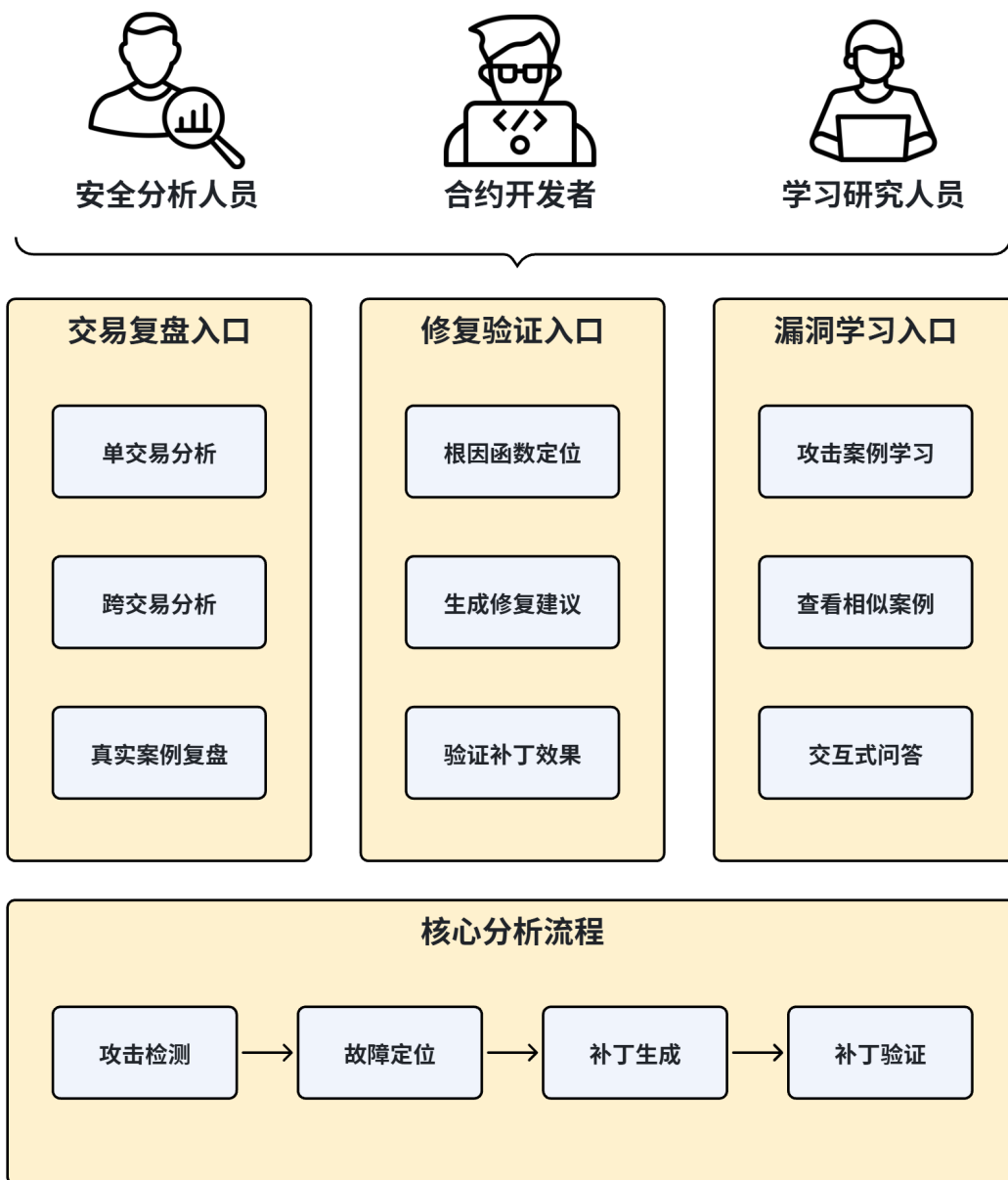


图 3 用户功能模块

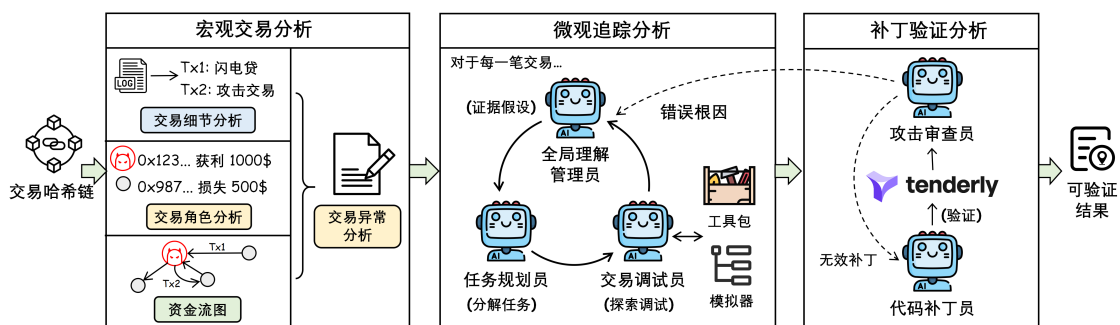


图 4 多智能体系统流程

Agent State

8

ACTIVE

94

EVENTS

GlobalMemory Administrator

12

Transaction Judge

4

Code Patcher

4

Task Organizer

12

Transaction Debugger

50

TxFaultAgent

4

TxRoleAgent

4

TxDetailAgent

4

系统状态

已完成

所有 Agent 正常运行

结构化报告

攻击复盘

一致性检查

学习导览

原始日志

智能体时间线

Structured Audit Report

Export MD

Export JSON

Raw Report

完整原始报告，保留模型输出上下文。

Final Fault Localization Report

1. Executive Summary

On March 17, 2022, an attacker executed a single-transaction exploit that stole **60,564 APE** (approximately **\$605,000** at the time of the attack) and **~14.15 ETH** from the **ApeCoin airdrop claim contract (AirdropGrapesToken)**. The attack leveraged the **NFTX BAYC Vault's zero-fee ERC-3156 flash-loan** to temporarily acquire the required NFTs, claimed the airdrop, and returned the NFTs within the same atomic operation. The only capital committed by the attacker was a single BAYC NFT (tokenId 1060).

Contracts involved:

Victim:

AirdropGrapesToken (0x025c...a1f) – the ApeCoin airdrop claim contract.

Exploited liquidity pool:

NFTX BAYC Vault (0xea47...e5c5) – a BAYC fractionalization vault that enables flash-loans.

Attacker-deployed:

Master router (0x3EbD...54f3) and vault (0x7797...13b).

Vulnerability type:

Flash-loan-based airdrop claim due to missing ownership verification (logic / calculation error). The claim contract checked only the current `balanceOf` of the caller, with no snapshot or holding period, allowing temporary NFT ownership to trigger a full airdrop payout.

Patch verification

has confirmed the root cause: a modified `claimTokens()` function that restricts callers to externally-owned accounts (`msg.sender == tx.origin`) caused the attacker's transaction to **revert** with the message `Contracts not allowed`, preventing any profit.

2. Root Cause Analysis

2.1 Core Vulnerability Mechanism

The `AirdropGrapesToken.claimTokens()` function determined eligibility exclusively from the **current** NFT balance of the caller (`msg.sender`). It performed **no** verification of:

How long the caller had held the NFTs

图 7 分析复盘区